



UEA

UNIVERSIDAD
ESTATAL AMAZÓNICA

Unidad 1

Algoritmos y Diagramas
de flujos

Tema 1

Diseño de Algoritmos

Subtema 2: Etapas del
diseño
algorítmico. Análisis,
diseño y
evaluación

Etapas del diseño algorítmico

Introducción a las etapas del diseño algorítmico: análisis, diseño y evaluación.

Estas etapas se complementan para crear algoritmos efectivos y soluciones de calidad.



Análisis algorítmico

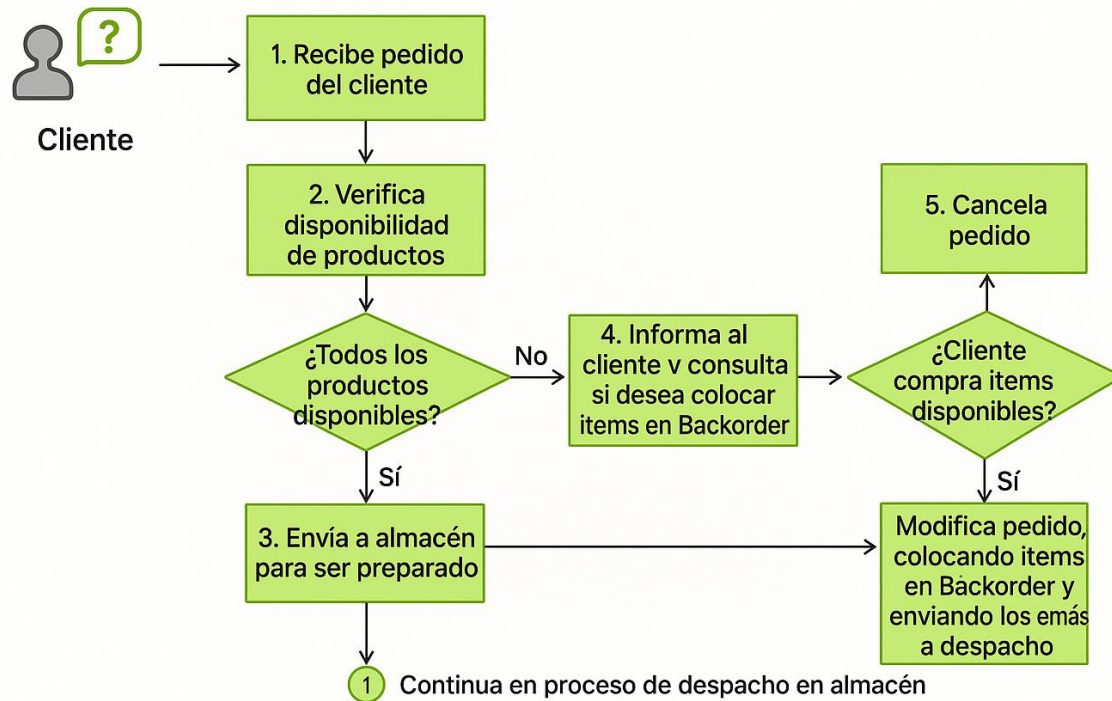
El análisis algorítmico es una etapa fundamental en el diseño de algoritmos.

Consiste en estudiar y comprender los requisitos del problema que se desea resolver.

Durante el análisis, se identifican las entradas necesarias para resolver el problema, las salidas que se deben obtener y las restricciones que se deben tener en cuenta



Proceso de pedido de venta



Para realizar un análisis algorítmico efectivo, se utilizan técnicas como el análisis de dominio, el análisis de requerimientos y el análisis de casos de uso

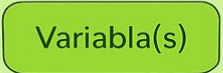
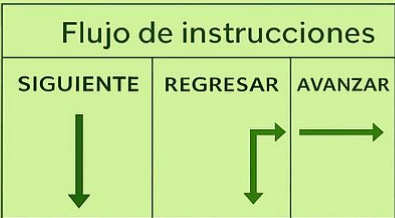
Análisis de Dominio:

El análisis de dominio implica estudiar el contexto y las reglas del problema, así como identificar las características de los datos involucrados.

Análisis de Requerimientos:

El análisis de requerimientos se centra en comprender las necesidades y expectativas de los usuarios y las limitaciones del sistema



PARTE DEL SISTEMA		DIAGRAMA DE FLUJO
Inicio		
Entrada		
Proceso		
		
	Calcular	
	Llamar a subproceso	
	Comparar	
	Retroalimentación	
	Etiquetas	 
	Salida	
Fin	Terminar	

Análisis de casos de uso:

Se utiliza para identificar los diferentes escenarios en los que el algoritmo debe funcionar correctamente.

Estas técnicas ayudan a definir claramente los objetivos del algoritmo y a comprender las complejidades y desafíos específicos del problema.

Durante el análisis, se pueden utilizar herramientas como pseudocódigo y diagramas de flujo para representar y comunicar la lógica del algoritmo.

TÉCNICAS O ENFOQUES EN EL DISEÑO ALGORÍTMICO

Enfoque descendente (top-down)

En este enfoque, se parte de una visión general del problema y se descompone en subproblemas más pequeños y manejables



Enfoque ascendente (bottom-up)

En este enfoque, se parte de soluciones individuales para subproblemas y se van combinando para construir soluciones más grandes y complejas.



Enfoque orientado a Objetos

Este enfoque se basa en la programación orientada a objetos (POO) y se centra en la identificación de objetos y sus interacciones para resolver el problema.



Enfoque basado en patrones

Este enfoque se basa en identificar y aplicar patrones de diseño comunes para resolver problemas recurrentes. Los patrones de diseño son soluciones probadas y documentadas.



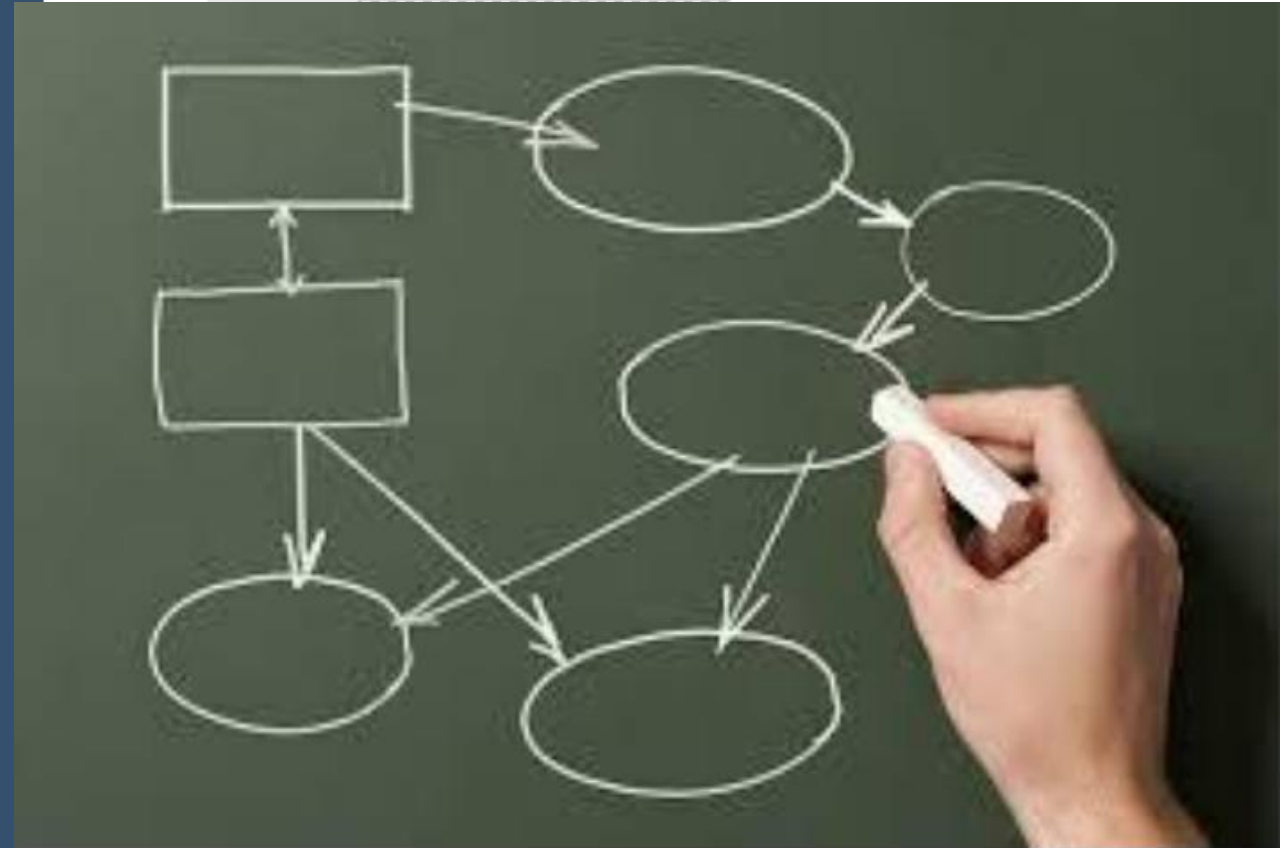
EVALUACIÓN DEL DISEÑO

- La evaluación del diseño algorítmico es un paso fundamental para garantizar la calidad y eficiencia del algoritmo desarrollado.
- Consiste en analizar y evaluar el diseño desde diferentes perspectivas para identificar posibles mejoras, corregir errores y verificar que cumple con los requisitos establecidos.

EVALUACIÓN DEL DISEÑO

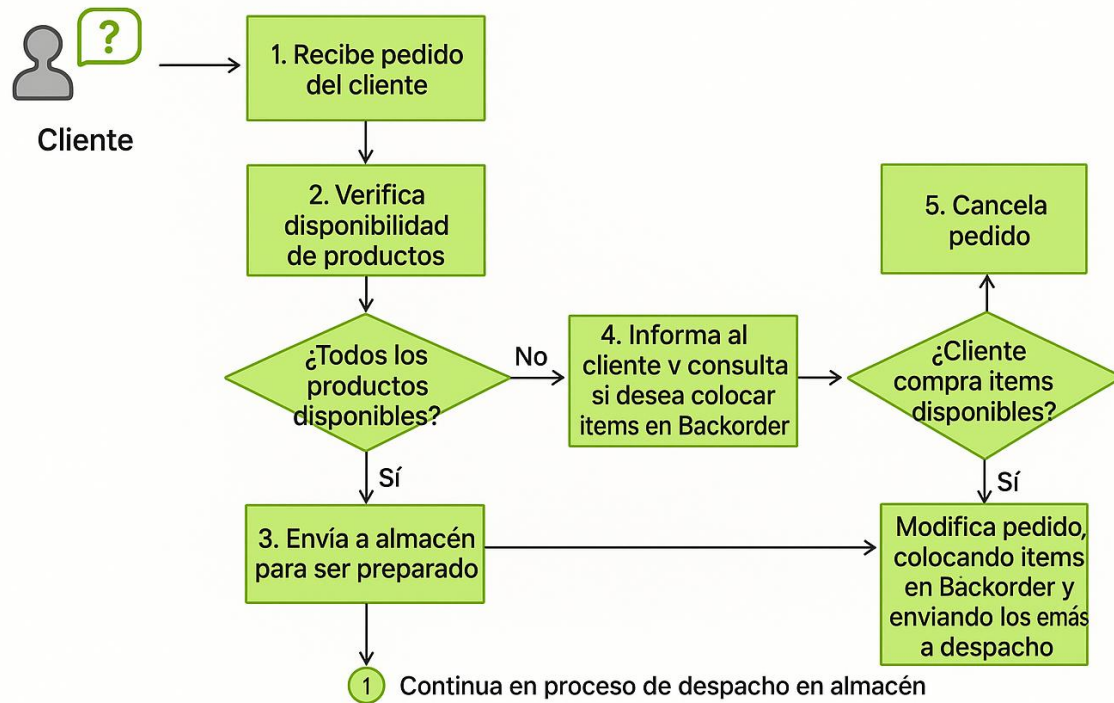
Aspectos clave

Correctitud: Se verifica que el algoritmo resuelve el problema planteado de manera precisa y exacta, produciendo los resultados esperados para diferentes casos de prueba. Se realizan pruebas exhaustivas para confirmar que no existen errores lógicos o de funcionamiento.



Eficiencia

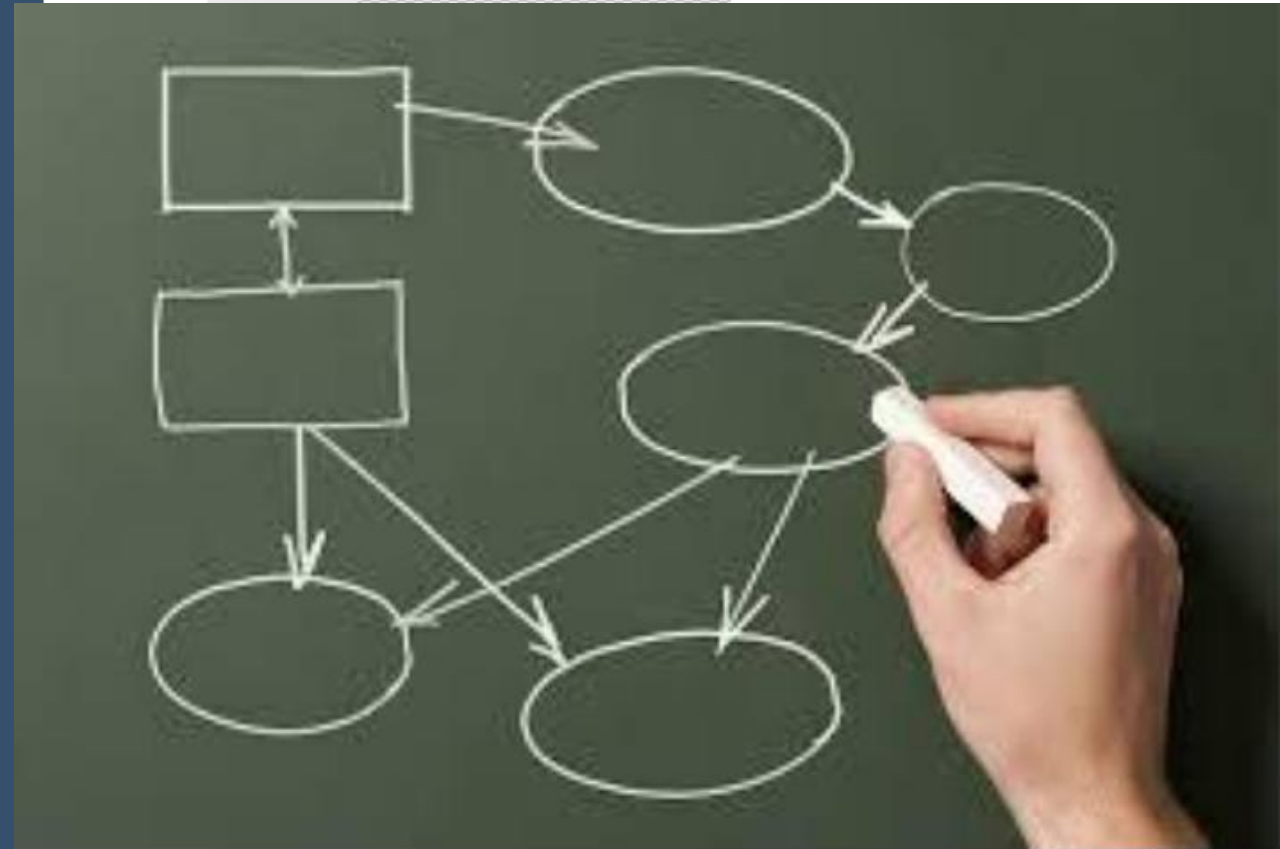
Proceso de pedido de venta



Se analiza el rendimiento del algoritmo en términos de tiempo y espacio requeridos para su ejecución. Se busca minimizar el tiempo de ejecución y la cantidad de recursos utilizados, optimizando la complejidad algorítmica.

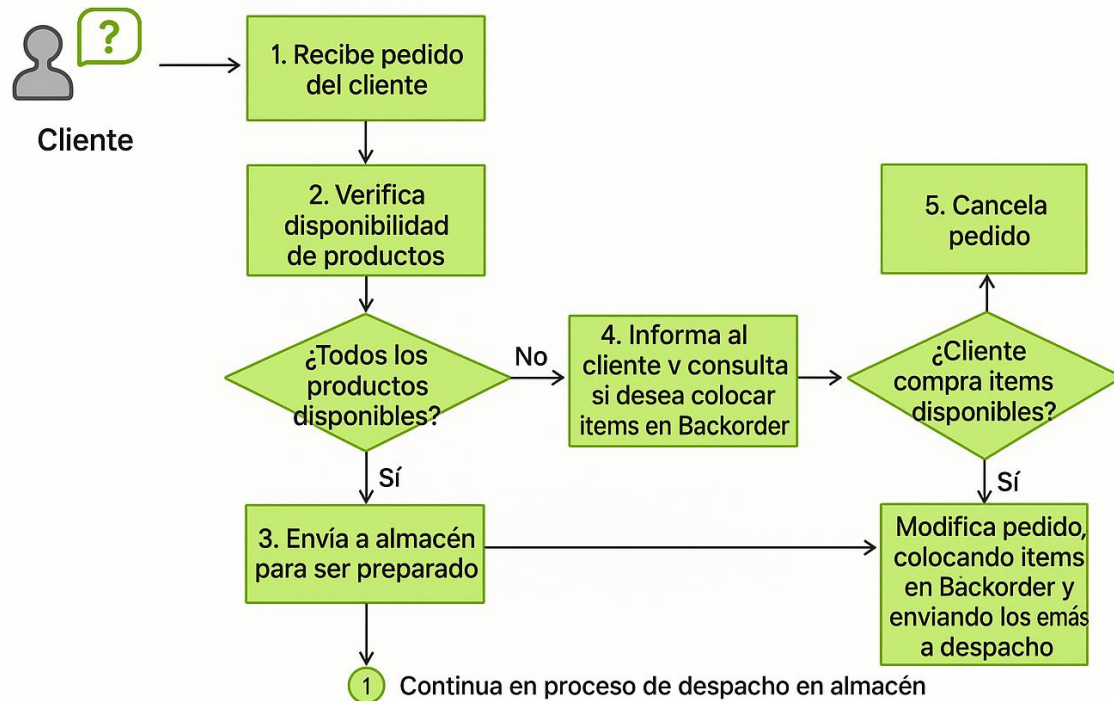
Mantenibilidad

Se evalúa la facilidad de entender, modificar y dar mantenimiento al algoritmo en el futuro. Se busca que el diseño sea modular, estructurado y legible, con comentarios claros y convenciones de nomenclatura adecuadas. Se considera la reutilización de código y la escalabilidad del algoritmo.



Robustez

Proceso de pedido de venta

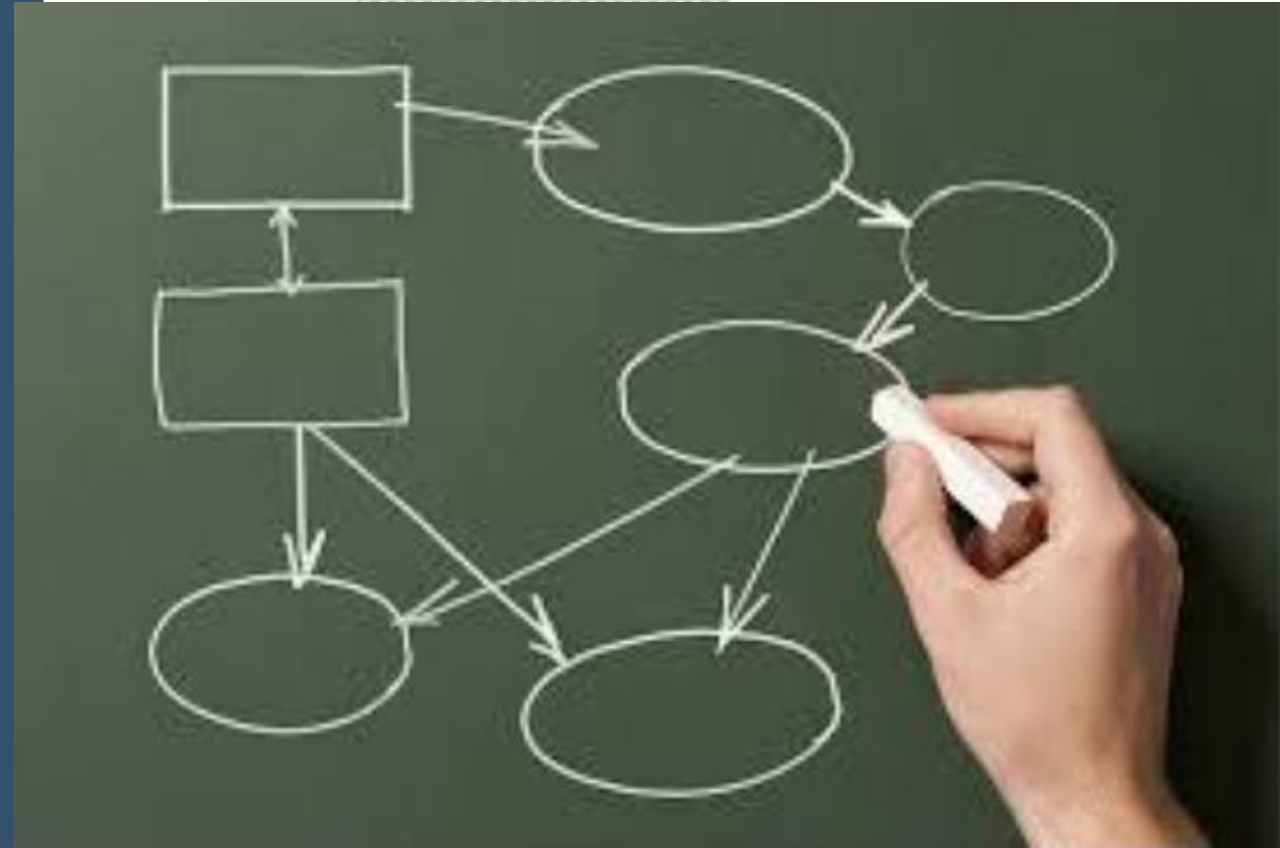


Se analiza la capacidad del algoritmo para manejar situaciones inesperadas, como entradas inválidas o datos atípicos.

Se buscan posibles vulnerabilidades o puntos débiles en el diseño y se implementan mecanismos de validación y manejo de errores.

Cumplimiento de requisitos

Se verifica que el diseño cumple con los requisitos establecidos en el enunciado del problema. Se comparan los resultados obtenidos con las especificaciones y se evalúa si se han considerado todas las restricciones y casos particulares mencionados.



Pruebas de algoritmos

- Las pruebas de algoritmos son un componente esencial en el proceso de desarrollo de software, ya que permiten verificar si un algoritmo produce los resultados esperados y se comporta de manera adecuada en diferentes escenarios.
- Estas pruebas se realizan con el objetivo de detectar posibles errores, garantizar la correctitud del algoritmo y comprobar su eficiencia en términos de tiempo y espacio.
- La evaluación del diseño algorítmico es un paso fundamental para garantizar la calidad y eficiencia del algoritmo desarrollado.

Pruebas de algoritmos

Casos de pruebas positivas

Son pruebas en las que se ingresan datos de entrada válidos y se espera obtener resultados correctos. Estos casos de prueba evalúan el comportamiento normal del algoritmo y ayudan a confirmar su correctitud.



Casos de pruebas negativas

Son pruebas en las que se ingresan datos de entrada inválidos o inadecuados y se espera que el algoritmo maneje adecuadamente estas situaciones. Estas pruebas evalúan la robustez y capacidad de manejo de errores del algoritmo.



Casos de prueba Límite

Son pruebas en las que se utilizan valores extremos o situaciones límite para evaluar cómo responde el algoritmo. Estas pruebas permiten identificar posibles problemas de desbordamiento, errores de redondeo u otros comportamientos inesperados.



Casos de prueba de rendimiento

Son pruebas que evalúan la eficiencia del algoritmo en términos de tiempo y recursos utilizados. Estas pruebas pueden involucrar conjuntos de datos grandes o repetir la ejecución del algoritmo muchas veces para medir su rendimiento en situaciones de carga.



Análisis de eficiencia

1. Complejidad temporal: Se refiere al tiempo que tarda el algoritmo en ejecutarse en función del tamaño de los datos de entrada. Se analiza la complejidad temporal para determinar si el algoritmo es eficiente y si cumple con los requisitos de tiempo establecidos.
2. Complejidad espacial: Se refiere a la cantidad de memoria o espacio requerido por el algoritmo para almacenar los datos y las estructuras de datos auxiliares. Se analiza la complejidad espacial para evaluar la eficiencia en el uso de recursos de memoria.
3. Análisis de casos límite: Se realizan pruebas con casos de entrada que representan situaciones extremas o límites para verificar si el algoritmo maneja correctamente estas situaciones y si su rendimiento se mantiene dentro de los límites aceptables.
4. Comparación de algoritmos: Se comparan diferentes algoritmos que resuelven el mismo problema para determinar cuál es más eficiente en términos de tiempo y espacio. Esto permite seleccionar el algoritmo más adecuado para una situación específica.

Refinamiento del diseño

1. Simplificación del algoritmo: Se busca simplificar el algoritmo eliminando pasos innecesarios o redundantes. Esto puede lograrse mediante la optimización de estructuras de datos, eliminación de bucles redundantes o la simplificación de condiciones lógicas.
2. Uso de técnicas de optimización: Se exploran técnicas específicas para mejorar la eficiencia del algoritmo, como el uso de algoritmos de búsqueda más eficientes, algoritmos de ordenación optimizados o técnicas de programación dinámica.
3. Evaluación del impacto de cambios: Se analiza el impacto de cualquier cambio realizado en el diseño del algoritmo. Esto implica evaluar cómo afecta a la eficiencia, corrección y modularidad del algoritmo en general.
4. Iteración y mejora continua: El proceso de refinamiento del diseño es iterativo. Se revisa y mejora el diseño inicial varias veces hasta lograr una solución óptima. Esto implica probar diferentes enfoques, realizar pruebas exhaustivas y recibir comentarios para realizar ajustes y mejoras adicionales.

Importancia del diseño algorítmico

Un diseño algorítmico adecuado es esencial para desarrollar software eficiente, mantenible, comprensible y escalable. Un buen diseño de algoritmos contribuye a la optimización de recursos, mejora la calidad del software y facilita su evolución a lo largo del tiempo.



***JUNTOS TRANSFORMAMOS
EL MUNDO DESDE
LA AMAZONÍA***

